

Advanced Intrusion Detection Systems

Richard Hoffmann
2439624

richard.hoffmann@stud.tu-darmstadt.de

Anh Duong Nguyen
2520766

duong.nguyen1@stud.tu-darmstadt.de

Benedikt Frisch
2552381

benedikt.frisch@stud.tu-darmstadt.de

1 Introduction

In today's digital age, cybersecurity is a critical concern for organizations and individuals alike. With the increasing reliance on computer networks and internet-based services, ensuring the preservation of the Confidentiality, Integrity, and Availability (CIA Triad) within cybersecurity has received enormous concern. Protecting our infrastructures from unauthorized access and malicious attacks is more and more important due to the rapidly changing and expanding of these threats targeting both private and corporate data. Given the growing concern for system security on our services and networks, Intrusion Detection Systems (IDS) have been developed to address these challenges by monitoring network traffic and system activities to detect suspicious behaviors

1.1 Definition and basic principles of IDS

In this section, we will provide a general overview of IDS and discusses the motivation underlying our work.

An Intrusion Detection System (IDS) is a security technology to observe network traffic and system activities in order to recognize suspicious behavior. It is an important part of modern cybersecurity infrastructure, aiming to identify intrusions before they can cause significant damage. The motivation behind using IDS lies in the challenges posed by modern software environments. Nowadays, software often consist of thousand or even million lines of codes, making manual inspection for malicious behavior impractical and highly inefficient. Human analysts are no longer capable of keeping pace with the growth of volume and complexity of data that must be analyzed. By implementing IDS, organizations can speed up threat detection and significantly reduce manual efforts.

Within the scope of this work, host-based Intrusion Detection Systems (HIDS) is considered. HIDS are deployed as software agents on the host systems. Their main function is to observe and analyze internal activities on the device, such as file access, process execution and system calls, enabling them to detect threats that are not visible at network level.

1.2 Methods of IDS

Intrusion Detection Systems (IDS) can be broadly categorized based on how they analyze data. The three main methods are anomaly-based, signature-based, and hybrid IDS, each with their own advantages and limitations.

Anomaly-based IDS focus on identifying deviations from normal behavior within a network or system. A baseline is established for normal activity by statistically analyzing network traffic or system activity over time. Potential incursions are indicated by notable departures from these baselines.

Signature-based IDS Signature-based IDS use a predefined database of attack signatures to identify malicious activity. If any signature is matched, this will trigger an alert, which indicates a potential intrusion attempt.

Hybrid IDS combine anomaly-based and signature-based detection methods. Hybrid IDS take advantage of signature-based detection to identify threats and advantage of anomaly-based detection to identify novel attacks and reduce false positives. This improves intrusion detection's overall efficiency.

1.3 What are PIDS

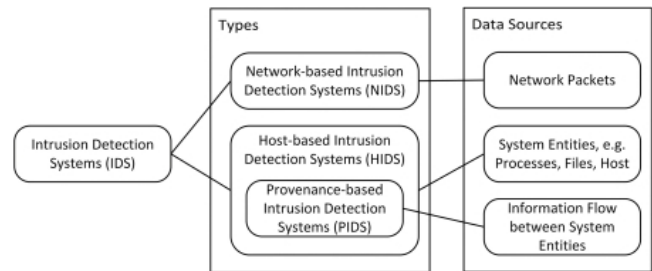


Figure 1: Types of IDS ¹

"Provenance" in Provenance-based Intrusion Detection Systems (PIDS) refers to two terms: Data Provenance and Provenance Graph. **Data Provenance** is a written record that

¹Provenance-based Intrusion Detection Systems: A Survey

shows the origin of an event and explains how and why the event came to be in its present state. The **Provenance Graph**, typically represented as a directed acyclic graph (DAG) $G = (N, E)$, models the relationships between system entities and operations. In this graph, the nodes N represent system entities (such as files, processes, or users), and the directed edges E represent operations or data flows (such as read, write, or execute actions). Together, these components allow the PIDS to analyze the system behavior and detect suspicious or unauthorized activities.

1.3.1 Challenges of PIDS

PIDS make use of data provenance to analyze system entities, their properties, the causalities, and information flow of system entities in the provenance graph. Then it uses that information to detect intrusion. However, several challenges arise as PIDS evolve and are applied to real-world environments. First the increasing speed of data provenance collection in large enterprises can lead to explosive data growth, reaching terabyte scale. Second, many Rule-based Intrusion detection methods depend on a threshold that is manually adjusted for each specific scenario, which makes them less scalable. Third, many systems utilize only a small portion of contextual information. More used information could improve detection performance. Lastly most available benchmark datasets are either outdated or lack representation of real world benign system activities, making it hard to evaluate systems under realistic conditions.

1.3.2 Analyzed PIDS approaches

In this work, we focus on two prominent approaches used in modern PIDS: the **embedding-based** and the **rule-based** approaches.

In the embedding-based approach, the PIDS represents, propagates, and aggregates information about system entities, events, and data/control flow in the graph using numerical vectors. So instead of analyzing the raw graph, the systems use embedding techniques like word2vec, doc2vec, Graph Neural Networks (GNNs), or graph2vec to transform graphs or subgraphs into vectors that summarize node behavior, connectivity patterns, and temporal evolution.

The rule-based approach relies on human defined rules that map system entities and events to predefined semantic units. These units are then propagated along the information flow within the provenance graph to capture causal relationships.

1.4 What is Deep Learning

According to [14], Deep Learning (DL) is a more complex branch of Machine learning (ML), that addresses certain limitations of shallow neural networks-neural networks that have

a small number of hidden layers between the input and output. It has demonstrated significant impact across various domains, such as image analysis, natural language understanding, and speech recognition. However, this approach demands substantial computational resources, leading to high memory consumption. Moreover, it underscores the challenge that deep learning-based solutions are inherently resource-intensive, slow, and lack transparency. Metrics that we can use to evaluate the efficiency of an IDS: Accuracy, Performance, Completeness, False alarm rate, number of features with the additional metrics: Fault tolerance, and Timeliness.

2 Overall Structure of Provenance-based Intrusion Detection Systems

The selected papers are all about Provenance based Intrusion Detection Systems (PIDS).

In this section, we describe the typical PIDS' components to provide general background knowledge relevant to the topic. To begin with, it is essential to understand the components involved in a typical PIDS workflow. PIDS are typically composed of the following components:

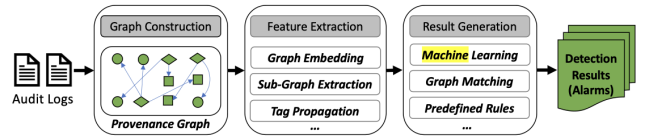


Figure 2: typical PIDS workflow ²

2.1 Graph Construction

Audit logs from the host are utilized to construct provenance graphs, where entities such as files, processes, and network connections are represented as nodes, and their interactions form the edges. Depending on the design of each PIDS, the scope of included events may vary: some systems focus narrowly on specific activity types, while others include broader execution contexts. To streamline analysis and reduce noise, certain systems also eliminate redundant or low-value edges during graph construction. Each PIDS employs its own tailored strategy for these steps, depending on its detection goals and system assumptions.

2.2 Edge Featurization

To enable graph learning, the provenance graph is transformed into a vectorized representation, this process is known as featurization. Systems convert symbolic elements into numerical

²Incorporating Gradients to Rules: Towards Lightweight, Adaptive Provenance-based Intrusion Detection

vectors that can be ingested by learning algorithms. The specific implementation of edge or node feature encoding varies across PIDS systems. Some approaches apply one-hot encoding, where each element is represented by a sparse binary vector, while others utilize embedding techniques such as Word2Vec or Doc2Vec, which map tokens to dense vectors that reflect contextual semantics. Word embedding models generate vector representations based on the co-occurrence and context of elements in the graph, enabling richer feature representation than purely symbolic methods. In contrast, one-hot encoding provides a simple, discrete mapping without capturing contextual similarity.

2.3 Graph Learning

During the graph learning phase, node and edge embeddings are iteratively updated by aggregating information from neighboring entities. This enables PIDS to capture both local and multi-hop behaviors within the provenance graph. Systems employ various architectures such as GNNs, transformers, or attention mechanisms, often trained with custom loss functions to reduce false positives. Some approaches also simplify graph structures during learning to enhance interpretability for system administrators.

2.4 Anomaly Detection

After learning patterns from the provenance graph, the IDS employs an automated mechanism to detect anomalies and generate alerts. These alerts enable system administrators to manage potential threats and conduct root cause analysis. Some systems, such as ORTHRUS and R-CAID, go a step further by offering automatic root cause attribution, helping analysts identify the origin and propagation path of an attack without manual inspection.

3 Comparison of chosen PIDSes

For our comparison eight state-of-the-art PIDSs are chosen. We will compare these systems step by step based on the aforementioned subsections in [Section 2](#).

3.1 Graph Construction

For the comparison of the Graph Construction we want to take a look at the used logging tools and whether the systems are agnostic to them. PIDS construct their graphs using logs generated by the system.

PROGRAPHER: PROGRAPHER considers a wide range of log events for graph construction, including those related to files, processes, network sockets, and users. Each event forms an directed edge between two entities (nodes), which represents the type of interaction and the direction of the information flow.

FLASH: FLASH begins the provenance graph construction with the transformation of system logs into provenance graph. This process contains three primary steps. Initially, FLASH parses systems logs, which document host-level activities including process executions, file operations, and network communications. Subsequently, it handles real-time log collection by grouping events into configurable batches, with the batch size K specifying the number of events per batch. Each batch is then processed sequentially and transformed into a provenance graph for further analysis.

The constructed provenance graph of FLASH consists of two types of nodes: process nodes and object nodes. Object nodes represent various system entities, including files, network flows, modules, and other objects. The edges connecting these nodes are labeled to indicate the type of event (system calls), illustrating the causal relationships between the connected nodes along with the corresponding event timestamps. Additionally, each node is enriched with attributes such as process name, command line, file path, IP address, port numbers, and module path, which provide essential context and details for analysis.

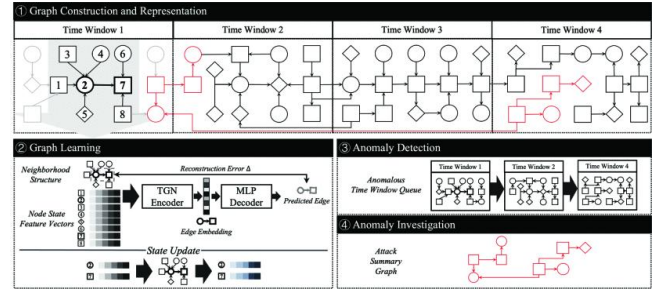


Figure 3: KAIROS workflow ³

KAIROS: KAIROS generates a complete provenance graph of the system based on audit logs gathered by logging frameworks. In KAIROS there exist three types of kernel objects, and nine types of interactions (i.e., types of the relationship between kernel subjects, objects, and the node attributes). The interactions, also called as events are transformed into a directed, time-stamped edge, inform about the source node and the destination node. The node's feature is encoded using a hierarchical feature hashing technique based on node attributes. This method takes high-dimensional input vectors and maps them into a lower-dimensional feature space while preserving the hierarchical similarity of the original input. KAIROS applies hierarchical feature hashing by encoding a node's attributes at multiple hierarchical levels. However the hierarchical feature hashing technique has assumption that two kernel entities of similar semantics have similar hierarchical features, which is a vulnerability allowing the adversary to manipulate an entity's attribute to evade detection. Nevertheless

³KAIROS: Practical Intrusion Detection and Investigation using Whole-system Provenance

KAIROS’ graph learning phase has its own countermeasure to come over these issues.

MAGIC: MAGIC first parses all log entries to extract system entities and interactions between them. If the log format already includes labels MAGIC uses them, otherwise it uses multi-label hashing to convert complex attributes into labels. In the Initial Embedding stage, MAGIC maps node and edge labels to fixed-size feature vectors using a lookup embedding. Same labels get mapped to the same vector, while different labels get different vectors. Since all labels come from the known log format, embeddings are created in a transductive setting without needing to handle unseen labels. Then the later described noise reduction gets applied.

ProcSAGE: ProcSAGE obtains its data from all audit logs and considers every syscall and relevant entity. The Graph consists of subjects and objects. Subjects refer to the processes or threads that perform actions, while objects are the things with which ones interact. The actions themselves, such as reading or writing, are represented as events. Each edge in the graph is equivalent to the information flow directed.

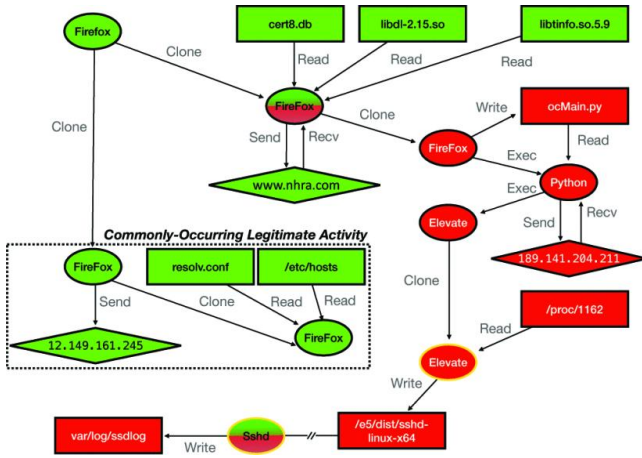


Figure 4: Graph constructed by R-CAID ⁴

R-CAID: R-CAID maps each vertex of its graph to a system entity, such as a file, process or network socket. Two vertices are considered connected by an edge if there exists a system call event between them.

A path exists between any two nodes if there is a series of edges linking them, where each edge is part of the overall set of edges in the system.

ORTHRUS: ORTHURS builds the graph from raw logs of different capture mechanisms, like CamFlow or Linux Audit. Processes, files, and network flows are extracted from the logs. Each process includes its name and command run, while the File logs contain the path of the file. For network flows, the local address, local port, remote address, and remote port are extracted. Event logs provide information about the subject,

object, event type, and timestamp.

CAPTAIN: When the system audit logs introduce a previously unseen system entity, it is added as a new node in the provenance graph and initialized with data tags. As events occur, these tags propagate along the edges between system entities, reflecting information flow. Predefined rules evaluate the tag propagation to determine whether an alarm should be generated.

Conclusion All systems start by turning system audit logs into provenance graphs with processes, files, and sockets as nodes and system calls as edges. This step is largely the same across the board. ORTHURS and FLASH enrich their graphs with additional context from multiple sources, which improves fidelity but increases complexity.

3.1.1 Redundancy Reduction

Some of the systems use techniques to compromise the size of the graph by removing redundant edges. They differ in their approach to do so:

PROGRAPHER: to handle the high volume of incoming logs, PROGRAPHER uses an time-ordered snapshot mechanism. As logs are processed, nodes and edges are getting added to the temporary cache graph and each edge gets an timestamp.

Once the number of nodes reaches the predefined size n , the

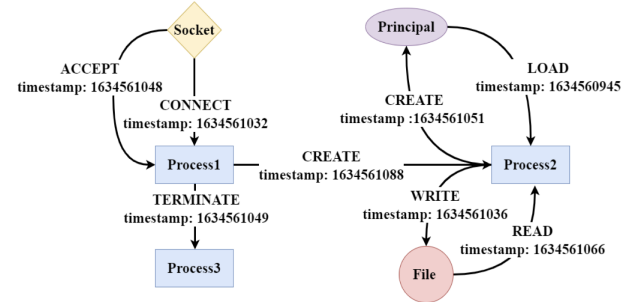


Figure 5: PROGRAPHER Timestamped Graph ⁵

graph is getting saved as a snapshot. As new logs arrive, older nodes are getting gradually removed based on a forgetting rate f , allowing the system to create overlapping snapshots with limited size. This overlap ensures that changes over time can be tracked, while the size of the graph is kept manageable.

FLASH: reduces redundancy by generating Attack Evolution Graphs (AEGs) from the large provenance graph and the alerts it produces. The key idea is to interlink causally related alerts within the provenance graph, constructing a concise sequence of AEGs. These graphs contain only alert nodes and their causal edges, significantly simplifying the original

⁴R-CAID: Embedding Root Cause Analysis within Provenance-based Intrusion Detection

⁵MAGIC: Detecting Advanced Persistent Threats via Masked Graph Representation Learning

graph structure. This reduction enhances clarity and enables security analysts to quickly understand the progression of an attack.

KAIROS: applies standard provenance graph reduction techniques to prune redundant elements from the anomalous sub-graph (queue q) while preserving its semantic integrity. This step reduces complexity without affecting the meaningful structure required for further analysis or attribution.

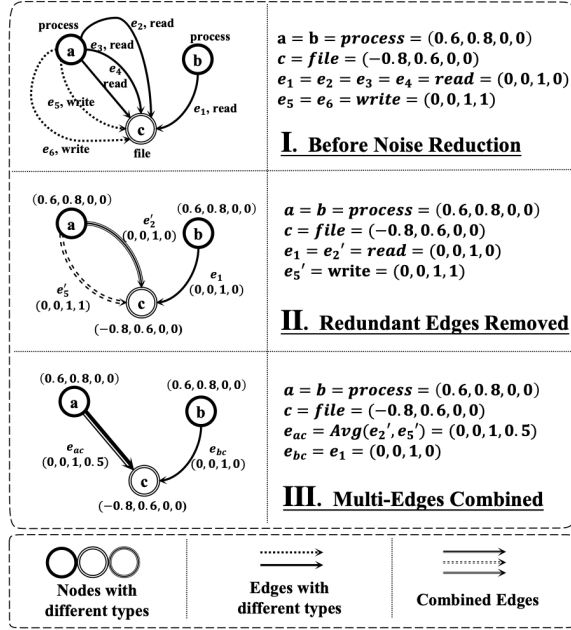


Figure 6: Noise Reduction in MAGIC ⁶

MAGIC: MAGIC operates in three steps when it comes to noise reduction. First redundant edges with the same label and embedding are getting removed, only one is kept. Then the remaining edges between a pair of nodes are getting combined into one single edge. As a final step, MAGIC combines edges with different labels by averaging their initial embedding. By performing these steps, MAGIC reduces the number of edges by an average of 79%, while keeping the graph semantically meaningful.

ProcSAGE: ProcSAGE does not rely on noise reduction techniques. Instead, it simplifies the provenance graph by breaking it down into smaller subgraphs based on syscall types. For instance, system calls related to process control (such as those used to create or manage processes) can be grouped into a dedicated subgraph. This approach makes it easier to identify patterns in how processes interact with one another.

R-CAID: no redundancy reduction.

ORTHRUS: uses Causality Preserved Reduction for edge pruning, by using capture systems such as CamFlow or

SPADE.

CAPTAIN: does not use any graph redundancy reduction technique.

Conclusion Here, systems diverge. ORTHRUS prunes edges while keeping causal consistency. ProcSAGE breaks graphs into syscall-level subgraphs, which aids efficiency but may lose context. FLASH transforms logs into Attack Evolution Graphs, easy for analysts but potentially overlooking subtle signals. MAGIC reduces redundancy through merging of edges and nodes, achieving scalability but at some cost in semantic richness. PROGRAPHER introduces temporal snapshots, which solve dependency explosion and improve scalability but may lose long-range dependencies. R-CAID and CAPTAIN do not focus on reduction.

3.2 Edge Featurization

We compare the different approaches for edge featurization. A distinction is made between embedding-based and rule-based systems.

3.2.1 embedding based PIDS

PROGRAPHER For Graph Learning PROGRAPHER uses an Encoder only technique employing the Graph2vec embedding model to capture key characteristics of the graph. Graph2vec uses the rooted subgraphs (RSGs) of every node as its input and then transform them with the help of the doc2vec embedding model into meaningful graph representations, which represent the categorical features of each RSG.

FLASH: FLASH employs a Word2Vec-based embedding technique to encode various node attributes found in provenance graphs, such as process names, command-line arguments, file paths, and IP addresses into semantically rich feature vectors. To address the lack of temporal context in standard embeddings, this technique is further modified to produce temporally sensitive embeddings, allowing the model to capture the sequence and timing of events more effectively.

KAIROS: bridges graph learning and community detection by using learned anomaly scores rather than relying on manually engineered features. It encodes node features using a hierarchical feature hashing technique based on node attributes, enabling efficient and semantically meaningful embeddings. KAIROS can also leverage other node embedding techniques, but these approaches make some assumptions about the similarity between two system entities.

MAGIC: To convert the graph into a series of vectors, MAGIC uses a lookup embedding. This creates a simple one to one mapping between a label and a fixed-dimensional vector. For example two nodes with the label "process" will receive the same embedding vector, whereas "process" and

⁶MAGIC: Detecting Advanced Persistent Threats via Masked Graph Representation Learning

"file" will be embedded differently.

ProcSAGE: ProcSAGE distinguishes between so called semantic and topological features for the graph embedding. The semantic part captures how frequently a process performs various system calls, reflecting its behavior over time. The topological part includes structural metrics such as the average degree of neighboring nodes and the local clustering coefficient, which describe how the process interacts with other entities in the system. These features are concatenated into a vector that serves as the input to the GNN "GraphSAGE" model.

R-CAID: R-CAID encodes the system identifiers gathered from the Audit Logs. R-CAID considers each file path of the training dataset a single sentence and passes it into Doc2Vec. Doc2Vec then groups similar sentences together and provides an embedding associated with each file path as the initial nodes feature. Additionally R-CAID employs One hot embedding for file paths. One-hot encoding transforms file paths into a numerical representation by mapping a single numerical value to each distinct path component.

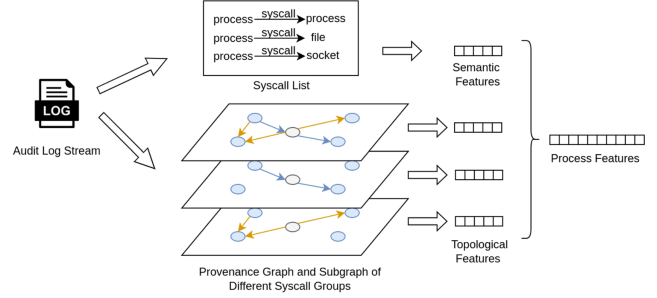


Figure 8: Edge Featurization in ProcSAGE⁸

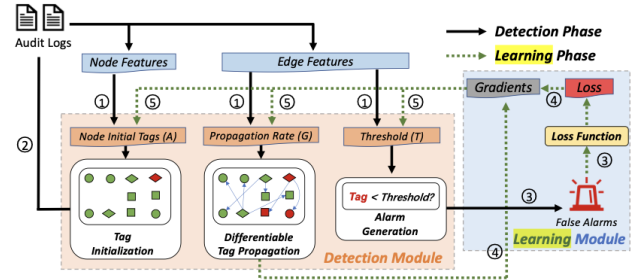


Figure 9: Overall framework of CAPTAIN⁹

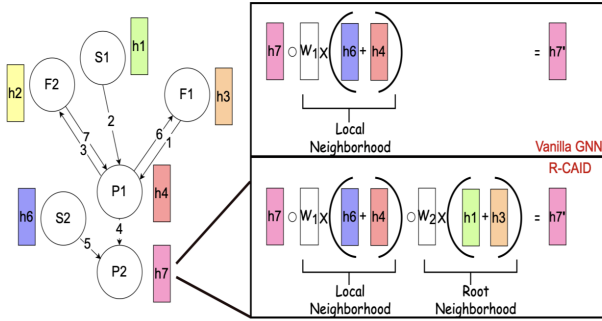


Figure 7: Embedding of an R-CAID node⁷

ORTHRUS: ORTHRUS uses the Word2Vec embedding model to transform textual attributes into a sequence of words. The model is trained on the corpus of all textual attributes, embedding each word w_i into a fixed-sized vector w_i that captures the similarity between words based on their context.

3.2.2 Rule-based PIDS

CAPTAIN: There are two phases in the overall framework of CAPTAIN

1. Detection phase: CAPTAIN determines whether to generate alarms based on the predefined rules, more specific CAPTAIN is a fine-grained rule-based PIDS.

- The node features: depend on the information granularity of audit logs, such as file path, process name, command line, IP address, port, etc.

- Each node in the provenance graph has its own data tag represented as a numerical vector $\langle c, i \rangle$, where c is the confidentiality score and i is the integrity score of the data; c and i are in $(0, 1)$.

- On the process nodes has additionally their code tag $\langle p \rangle$ used to indicate the integrity of the code.

- There also exist three core parameters, so that we can use to propose the corresponding propagation rules, alarm triggering rules, and these parameters can then be used to achieve more flexible rule-based detection.

- The nodes in the provenance graph of CAPTAIN are also initialized with the following three parameters, which are the core of the fine-grained rule-based PIDS:

- Tag Initialization Parameter A ,
- Tag Propagation Rate Parameter G ,
- Alarm Generation Threshold Parameter T .

⁷A. Goyal, G. Wang and A. Bates, "R-CAID: Embedding Root Cause Analysis within Provenance-based Intrusion Detection," 2024 IEEE Symposium on Security and Privacy (SP), San Francisco, CA, USA, 2024, pp. 3519

⁸Xu, B., Gong, Y., Geng, X. et al. ProcSAGE: an efficient host threat detection method based on graph representation learning. Cybersecurity 7, 51 (2024), p.7

⁹Incorporating Gradients to Rules: Towards Lightweight, Adaptive Provenance-based Intrusion Detection

- The edge features: While CAPTAIN does not use learned edge embeddings, edges play a critical role in determining how tags are propagated. Each edge type (e.g., read, write, exec) is linked to a specific propagation rule. The propagation process G determines how influence flows and whether conditions for alarm T are met.

2. Learning phase: also known as graph learning, which we will view later in Section 3.3.

Conclusion Here the differences become larger. ORTHRUS and FLASH both use an Word2Vec-based embedding mechanisms, while R-CAID uses a mix of Doc2Vec and one-hot encodings, enhanced with root-cause overlays for causal context. MAGIC uses simple lookup embeddings, efficient but not very rich. KAIROS applies hierarchical feature hashing, while ProcSAGE combines semantic and topological features of the graph. CAPTAIN uses no embedding but rule-based tags, which include integrity and confidentiality of a node.

3.3 Graph Learning

The different models use following approaches for Graph Learning:

PROGRAPHER: For Graph Learning PROGRAPHER uses an Encoder only technique employing the Graph2vec embedding model to capture key characteristics of the graph.

Graph2vec uses the rooted subgraphs (RSGs) of every node as its input and then transform them with the help of the doc2vec embedding model into meaningful graph representations, which represent the categorical features of each RSG. This allows to collect a nodes neighborhoods information, which are essential information for anomaly detection.

To get the RSG for every node in the graph, graph2vec utilizes the Weisfeiler Lehman (WL) graph kernel. The WL uses the label of the node itself, the label of its edges and all neighboring nodes and generates a new label, the RSG label, which contains the nodes neighborhood information.

This process is repeated d times to capture the neighborhood information of each node at increasing depths. In comparison to the original WL kernel, PROGRAPHER does not only consider node types as labels, but additionally also edge types, in order to better capture the structure of provenance graphs. Moreover, PROGRAPHER improves the efficiency of the RSG generation, by only keeping a unique label for each RSG, instead of multiple redundant ones like in the original WL implementation.

After the RSG generation the encoder then learns the embeddings for a snapshot S_i by maximizing the log-likelihood of the RSG of every node.

Through backpropagation and gradient descent, the embeddings of each snapshot S_i are optimized to generate an accurate mapping of key characteristics.

For each snapshot, the model randomly selects a set of negative RSG samples from the entire subgraph collection and a set of positive RSG samples, while ensuring that the two sets are disjunctive. The objective function is then modified to simultaneously maximize the likelihood of the positive sample set, while minimizing the likelihood of the negative sample set.

FLASH: FLASH employs Provenance Graph Representation Learning to detect Advanced Persistent Threats (APTs). To begin, a modified Word2Vec technique is used to embed node features such as process names, file paths, IP addresses, and command-line arguments. These feature vectors serve as inputs to a GNN, which learns not only the semantic properties of nodes but also their structural context within the provenance graph. While this approach enables high detection accuracy, it introduces challenges such as resource overhead and scalability limitations, particularly during training on large-scale graphs. To address this, FLASH introduces two optimizations:

1. Selective Graph Traversal:

Inspired by the GraphSAGE framework, this technique enhances efficiency by guiding the GNN to selectively aggregate information from relevant portions of the graph, rather than traversing the entire graph exhaustively. This is achieved by assigning priorities to specific edge types that are more relevant for threat detection and excluding irrelevant or redundant subgraphs. As a result, only a subset of nodes and edges are used during aggregation, significantly reducing computational cost.

2. Semi-Supervised Node Classification with Weighted Cross-Entropy Loss:

This is part of a semi-supervised node classification approach used to train the described GNN model. This uses the node's input features and graph structure to classify its type such as process, file, and socket. Using the issue that the node types in the system logs are imbalanced, the weighted cross-entropy is used as the loss function, this function addresses the issue by assigning a weight to each class that reflects its importance. Generally, the weight assigned to each class is inversely related to how frequently that class appears in the data; so that it ensures that the loss function pays attention to the minority classes, then enable FLASH to improve the model's competence in correctly classifying those classes. The weighted cross-entropy loss is defined as followed:

$$L(y, f(x)) = - \sum_{i=1}^n (w_i y_i \log(f(x)_i) + (1 - y_i) \log(1 - f(x)_i))$$

where y is the true label vector, $f(x)$ is the predicted label vector, n is the number of classes, w_i is the weight for the i -th class, and $\log$ is the natural logarithm. Once

the model is trained, this GNN can be used as input to FLASH' lightweight classifier, since it already can learn the structural relationships between different node types in the provenance graph, i.e., efficiently identify stealthy attack nodes by leveraging the power of GNN and a lightweight classifier, without the need for continuous GNN execution during live intrusion detection.

KAIROS: KAIROS applies fine-grained, temporal-spatial graph learning to monitor system behavior at runtime across large-scale, distributed provenance graphs. Its graph learning approach is built on an encoder-decoder architecture: the encoder generates embeddings based on an edge's neighborhood structure and the states of its source and destination nodes, while the decoder attempts to reconstruct the original edge from this embedding.

We call the difference between the original edge and the reconstructed edge the "reconstruction error", which then in the training phase, KAIROS simultaneously trains the encoder and the decoder to minimize reconstruction errors of benign edges. In addition, the reconstruction error of each edge serves as a basis for anomaly detection and subsequent investigation during deployment. Furthermore, the states of the source and destination nodes of the edge will be updated.

KAIROS captures both temporal (interactions between an entity and other entities) and structural (sequences of events involving an entity) relationships in a provenance graph. These relationships offer valuable contextual information that allows us to characterize normal system behavior and differentiate it from anomalies. The temporal information can also be used to reveal subtle behavioral differences, which can not be detected by comparing graph structures, and static snapshots of a dynamic provenance graph.

The goal of training is to minimize the difference between the actual edge type and the type predicted by the decoder from its embedding.

MAGIC: In MAGIC, embeddings are updated through a self-supervised learning process using Graph Attention Networks (GAT). The key idea is that each embedding is refined by looking at the surrounding context in the graph, especially the embeddings of neighboring elements.

During training, MAGIC randomly masks some embeddings, and the model learns to reconstruct the missing information using only the visible parts of the graph. This forces the model to capture deeper patterns in how elements relate to one another.

The update process works in multiple steps, called layers. At each layer, an embedding is adjusted by aggregating information from its neighbors, but not all neighbors are treated equally. MAGIC uses an attention mechanism to decide how much influence each neighbor should have. More relevant neighbors get higher weights, while less relevant neighbors get lower weights. This allows the model to focus on important interactions.

As information flows through the layers, each embedding

evolves to reflect not only its own characteristics but also the multi-hop context around it. After several layers, the model outputs an embedding that encodes both local behavior and broader structural patterns.

ProcSAGE: ProcSAGE uses a type of Graph Neural Network (GNN) to learn structural and contextual patterns of processes within the provenance graph. The model updates each node's representation by aggregating information from its neighbors in the graph. In each layer of the network, a node collects feature information from its adjacent nodes, aggregates it, and combines it with its current representation to produce a new, more informative embedding.

This neighborhood aggregation is performed over multiple layers, allowing each node to incorporate information from its broader surroundings, not just immediate neighbors. As training progresses, the model learns to position similar processes close together in the embedding space, while pushing anomalous ones further apart.

Conclusion Most of the system rely on GNNs or related models. Specially, R-CAID strengthens GNN by integrating an root-cause layer, making it resilient to mimicry attacks but computationally heavier. ProcSAGE uses GraphSAGE, which is simpler and efficient, while MAGIC depends on Graph Attention Networks, which are heavier.

3.4 Detailed Graph Learning Comparison

The following section will include a detailed comparison of the graph learning approaches of R-CAID, ORTHRUS and CAPTAIN.

3.4.1 R-CAID

Graph Let $G = (V, E)$ be the provenance graph. Each vertex $v_i \in V$ is a system entity (e.g. process or file) and each directed edge $e = (v_s, v_d, rel) \in E$ encodes a system event or dependency (e.g. process v_s writes to file v_d). The graph is typically acyclic, but system behavior can introduce cycles, which are resolved via timestamps in the construction.

For each node v_i , we associate an embedding that is refined layer by layer. The intermediate representation of a node v_i after l layers is noted as $x_i^{(l)}$.

Local Neighborhood and Basic GNN Propagation Define the K -hop local neighborhood N_i of v_i as a set of ancestor nodes reachable within K hops. Standard GNN message passing updates a node by aggregating over its local neighborhood and combining that with its previous representation. In abstract form, one layer of update for node v_i is:

¹⁰ A. Goyal, G. Wang and A. Bates, "R-CAID: Embedding Root Cause Analysis within Provenance-based Intrusion Detection," 2024 IEEE Symposium on Security and Privacy (SP), San Francisco, CA, USA, 2024, pp. 3519

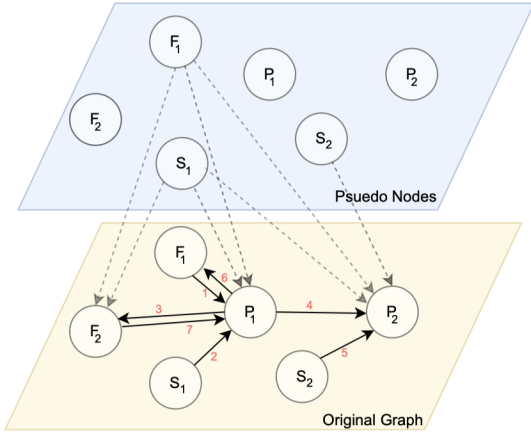


Figure 10: Graph-structure in R-CAID ¹⁰

$$x_i^{(l)} = h^{(l)}(U(x_i^{(l-1)}, A(N_i^{(l-1)})))$$

- $A(N_i^{(l-1)})$ aggregates the representations of the neighboring vertex.
- U is an update function (often concatenation and MLP).
- h^l is a nonlinear transformation at layer l .

Stacking K such layers allows to propagate from nodes up to K hops away. The final embeddings after K layers form the matrix $E^* \in \mathbb{R}^{|V| \times D}$, where each row is the embedding of one node.

Root neighborhood augmentation R-CAID identifies for each node a set of root causes and constructs an additional root neighborhood overlay that supplies long-range causal context, which is not visible in normal GNN message parsing. The key idea is to expose each node not only to its nearby context, but also directly to its root causes, whatever graph distance it may have.

A node v_k is identified as a root cause if its first timestamped event in the provenance graph is an outgoing edge. For each such root v_k , R-CAID creates a pseudo-root node p_k that mirrors v_k 's initial features and prevents updates flowing back into it.

Then, for every descendant v_j such that there exists a path $P(v_k, v_j)$, an edge is added from p_k directly to v_j in the pseudo-graph G' . This constructs the root neighborhood of v_j , denoted R_j , as all pseudo-root nodes connected into v_j .

Combined embedding update with local and root neighborhoods R-CAID modifies the per-layer update to incorporate both the local neighborhood N_i and the root neighborhood R_i . At layer K , the embedding for v_i is:

$$x_i^{(K)} = h^{(K)}(U(x_i^{(K-1)}, C(A(N_i), A(R_i))))$$

- $A(N_i)$ aggregates the local neighbors representation
- $A(R_i)$ aggregates the root neighborhoods representation
- C denotes concatenation of the two aggregated vectors
- U combines the previous self representation with the combined neighbor context
- $h^{(K)}$ applies the layer-specific transformation

This can be seen as enriching standard message passing with an additional channel carrying long-range causal signals. The concatenation ensures that the model preserves a distinction between local context and root-origin context before blending them.

Root Paths R-CAID also provides the option extending the root neighborhood beyond the direct pseudo-roots to entire root paths of length L . Let P_i be the set of pseudo-root path nodes that are eventually connected into v_i . In this optional case, the update becomes:

$$x_i^{(K)} = h^{(K)}(U(x_i^{(K-1)}, C(A(N_i), A(P_i))))$$

with the upper mentioned functions and $A(P_i)$ as an aggregation function, which collects embeddings of the pseudo-root path nodes.

3.4.2 ORTHRUS

The ORTHRUS system relies on an encoder-decoder architecture that helps it analyze the temporal and spatial relationships between events in a provenance graph. Unlike traditional static graphs that only show spatial relationships, ORTHRUS uses a dynamic graph structure that includes timestamps for each node and edge. This allows the encoder to learn meaningful representations that capture both the structural patterns and the timing of events generated by system entities, improving the detection of temporal dynamics within a graph.

Encoder: ORTHRUS features a lightweight encoder that efficiently learns temporal edge embeddings, enabling it to detect anomalies in system events. Unlike other systems, such as KAIROS, ORTHRUS's encoder doesn't rely on state memory, which reduces memory usage and improves performance. Instead, it uses a temporal sampling approach to capture past events involving each system entity. This approach involves sampling the N most recent edges that interacted with a node, preserving temporal information of previous events.

The sampling function is defined as follows:

$$S_N(v) = \text{SAMPLES}(S_t(v), N, T, t),$$

where $S_t(v)$ is the set of all edges prior to time t , and T is the timestamp associated to the edges in E .

Additionally, the encoder also incorporates features of both system entities and events, such as system call types and entity types, to enrich the graph. This, together with the structural and temporal information about each node, allows the attention-based graph neural network (GNN) of ORTHRUS to effectively and efficiently capture patterns in the data, enabling it to detect potential threats.

Embeddings for nodes are calculated using the features of the current node itself, its neighboring nodes and the features of the edges:

$$h_v = GNN(x_v, x_u, e_{uv} | (u, v) \in S_N(v))$$

, with x_v being the current node, x_u the start node and e_{uv} the edge feature of the edge (u, v) .

For the GNN ORTHRUS employs the GNN introduced by Shi et al. Using attention, it is able to allow each node to focus on the most relevant neighboring information during aggregation, without creating long dependencies within the NN.

ORTHRUS calculates for every edge (u, v) an attention coefficient $\alpha_{u, v}$:

$$\alpha_{u, v} = \text{softmax}\left(\frac{(W_3 x_v)^T (W_4 x_u + W_5 e_{uv})}{\sqrt{d}}\right) \quad (1)$$

Each feature of the source and destination nodes, as well as the edge are weighted using learnable weight matrices W_3 , W_4 and W_5 . D is the hidden size. In the last step the softmax function is applied in order to normalize the result.

The attention coefficient together with the weight matrix W_5 is then used during the aggregation step to weigh the learned feature of the neighboring nodes:

$$h_v = W_1 x_v + \sum \alpha_{u, v} (W_2 x_u + W_5 e_{uv}) \quad (2)$$

Decoder: The decoder is trained to learn the embeddings by optimizing a loss function that predicts the type of edge between connected nodes. By accurately predicting edge types, ORTHRUS gains a deeper understanding of how system entities interact and relate to each other, which is crucial for identifying potential threats. The model's ability to predict event types is also improved by incorporating node and edge types as features, allowing it to consider the context in which events occur. This enables the model to better distinguish between harmless and potentially malicious events. ORTHRUS decoder differentiates between source and destination entities by using separate weight matrices during their linear transformations to their embeddings, which helps the model understand their different roles and semantics during training. ORTHRUS calculates the prediction of a type for an edge like the following:

$$y_{uv} = \sigma(W_g [W_s h_u, W_d h_v]) \quad (3)$$

with y_{uv} being the predicted edge type, $[..]$ the concatenation operation, σ the Sigmoid activation function and W_g , W_s and W_d the gate, source and destination weight matrices.

The prediction is then, together with the ground truth, feed into a Cross-Entropy (CE) loss function to produce a loss for the current training example. Minimizing the loss for every being edge in the training data, the weight within the encode and decoder are updated allowing ORTHRUS to learn the users normal activity.

3.4.3 CAPTAIN

Not like the embedding-based PIDS, which leverage the power of deep learning, CAPTAIN uses instead fine-grained rules. By this way it allows the rule-based detector to learn and adapt from the training data and as result, the detection accuracy is improved without the extensive resource demands and runtime overhead. In the end our system enhances the overall efficacy of rule-based PIDS but also maintain its operational simplicity and clarity.

The graph learning phase of CAPTAIN occurs in the second phase of the framework of CAPTAIN, where CAPTAIN refines tags' parameters introduced in detection phase to fulfill its detection objectives. The objectives of CAPTAIN are reduced to the optimizing problem, where Loss Function and Differentiable Detection framework, and Detection Loss are used to solve it, more specific they are used to fine-tune the parameters.

First of all, we need to know these following equations:

$$1. \text{tag}_{\text{dest}}^{\text{new}} = g_e \cdot \text{tag}_{\text{rule}} + (1 - g_e) \cdot \text{tag}_{\text{dest}},$$

where tag_{rule} can be defined with CAPTAIN's tag propagation policies table, and g_e is set to 1 for all edge $e \in E$ to make propagation fully effective

$$2. \text{detection function } f(e):$$

$$f(e) = \text{tag} - \text{thr}_e = \begin{cases} \text{malicious}, & \text{if } f(e) < 0 \\ \text{benign}, & \text{otherwise} \end{cases}$$

,where thr_e is the node's alarm threshold, and tag is the relevant tag on the node of interest on edge e . Before training, thr_e will be set to 0.5 for every edge.

Loss function: according to [11], we know that the false alarm term mainly focuses on penalizing erroneously triggered alarms. Specifically, an event e triggers a false alarm means $f(e)$ should be greater than 0 but it does not. Since $f(e) \in [-1, 1]$, we can use the Mean Squared Error $(y_e - f(e))^2$ as the loss function for all events that trigger a false alarm. y_e is set as 1 for those benign events, and -1 for the malicious events. Because we only use benign data for training, all y_e should be 1. The false alarm term in the loss function can be formalized as

$$L(e) = \max(0, (1 - f(e))^2 - 1)$$

CAPTAIN uses only a small group of parameter in the training phase, the One-class classification (OCC) algorithms

in the ML-area is applied, the formula of *Loss* is an example of this algorithm.

Detection Loss: a regularizer term is added in the Loss function to avoid the parameters becoming too lenient when the malicious training sample is absent. The essential thought is to make the adaptive parameters as close to the default values (A_0, G_0, T_0) as possible. By minimizing the l_2 distance between the adaptive parameters and the default values, CAPTAIN protects the detection capability during training. Therefore, the detection loss can be formalized as

$$Loss = \sum_{e \in E} \mathcal{L}(e) + \alpha \|A - A_0\|_2 + \gamma \|G - G_0\|_2 + \tau \|T - T_0\|_2$$

Differentiable Detection Framework: is used to adjust the adaptive parameters based on the Loss Function; this allows us to keep the gradients of the adaptive parameters with respect to *Loss*. We apply chain rule in Loss function for the parameter a_n :

$$\frac{\partial Loss}{\partial a_n} = \sum_{e \in E} \frac{\partial \mathcal{L}(e)}{\partial f} \cdot \frac{\partial f}{\partial a_n} + \alpha \cdot (a_n - a_0)$$

Now the challenge is to compute the gradients

$$\frac{\partial f}{\partial a_n}, \quad \frac{\partial f}{\partial g_e}, \quad \text{and} \quad \frac{\partial f}{\partial thr_e},$$

since the rule-based detection process is usually modeled as the branch selection based on rules. In CAPTAIN, the detection process is modeled as a differentiable function and the gradients of each adaptive parameter are calculated, i.e., if the tags are propagated, the corresponding gradients are updated and recorded, by this it enables the association between the adaptive parameters with the loss, which makes it possible to perform the gradient descent algorithm to find the optimal parameters that minimize the loss.

3.5 Anomaly Detection

PROGRAPHER: Each generated snapshot is evaluated by comparing the predicted embedding, generated through the RCNN, with the actual embedding. If the difference between the two representations is too large, the snapshot is flagged as anomalous. To give a more detailed attribution, PROGRAPHER identifies the subgraphs within the snapshot that are most responsible for the deviation. These subgraphs are then ranked and presented as the most suspicious components of the snapshot.

FLASH: XGBoost is chosen for anomaly detection task, XGBoost uses an ensemble of decision trees and advanced boosting techniques for enhanced accuracy; predicts the probability of sample classification using the following function:

$$f(x) = \sum_{k=1}^K f_k(x), \quad \text{where } f(x) = \text{sigmoid}(\alpha_k T_k(x)),$$

and $T_k(x)$ is the prediction of the k -th tree, and α_k determines the tree's contribution. And minimizes the regularized objective function $J = \mathcal{L}(y, f(x)) + \mathcal{R}(f)$, using gradient boosting to iteratively add new trees to the ensemble.

FLASH detects anomalous nodes by comparing predicted and actual node types. The model considers both neighborhood structure and node attributes to learn benign patterns associated with various node types.

The XGBoost model uses the concatenated Word2Vec encoding vectors and GNN embedding vectors per node. It retrieves GNN embeddings from the pre-trained key-value store, generates Word2Vec features in real-time, performs inference, and saves the output for the next pipeline stage. This robust process underpins our IDS's performance and scalability.

KAIROS: At a high level, KAIROS performs anomaly detection at the level of time windows. KAIROS can construct many time window queues, and each time window can belong to multiple queues as long as correlation exists between time windows in those queues.

Time window queues provide KAIROS with additional context necessary to distinguish attack behavior from unusual but benign activity and further, to reconstruct series of events constituting the attack.

Anomalous time window queues significantly reduce the size of the graph that sysadmins must inspect in case of an intrusion alarm, they can still contain thousands of nodes and edges.

MAGIC: For anomaly detection, MAGIC compares new embeddings to a memory of benign embeddings using a k -nearest neighbor search. In the training phase, MAGIC uses embeddings from only benign data and stores them in a so called a K-D Tree. This structure makes it possible to find embeddings that are close to each other quick and easy.

When a new embedding x is created during the detection phase, the module searches the K-D Tree to find its k nearest neighbors x_i that were saved from the benign training data. It then calculates the average distance between x and his neighbors:

$$dist_x = \frac{1}{|N_x|} \sum_{x_i \in N_x} \|x - x_i\|$$

Here $\|x - x_i\|$ denotes the Euclidean distance between the embeddings. The anomaly score is then calculated by normalizing this average distance against a reference distance $\bar{dist}$, which is the mean neighbor distance between all embeddings and its k -nearest neighbors during training phase:

$$score_x = \frac{dist_x}{\bar{dist}}$$

Finally, the detection decision is made by thresholding the

anomaly score:

$$\text{result}_x \begin{cases} 1 & \text{if } \text{score}_x \geq \theta, \\ 0 & \text{if } \text{score}_x < \theta \end{cases}$$

If $\text{result}_x = 1$, the embedding is marked as an outlier and may represent malicious behavior, if it is 0, it is considered as unsuspicious.

ProcSAGE: ProcSAGE first trains its model on data that contains only benign behavior. The embeddings of these processes are grouped into clusters, with each cluster representing a typical pattern of activity. When a new process is observed, its embedding is generated by the trained GraphSAGE model and compared to the existing cluster centers. If the embedding lies far from the nearest cluster center or shows a small difference between the distances to its first and second nearest clusters, indicating it does not clearly belong to any learned pattern, it is considered anomalous and flagged as a potential threat.

R-CAID: R-CAID assigns an anomaly score to each process node in the test set via three stages. First, a fully trained GNN maps every process node from both training and test sets to a D-dimensional embedding. Second, the IDS applies K-Means clustering to the training embeddings, selecting the best number of clusters K using the elbow method [46], which evaluates a range of K values and chooses the one that minimizes within- and between-cluster variance. Third, during testing each node of the test set receives an anomaly score based on the Median Absolute Deviation (MAD): the Euclidean distance between a test node and the median of its nearest cluster.

Then during detection, any node whose Euclidean distances deviate beyond a certain variation, compared to all the Euclidean distances of nodes in the test set, are flagged as anomalous.

ORTHRUS: ORTHRUS employs a two-step anomaly detection method. First, it applies an automatic thresholding technique to detect abnormal nodes. When each assigned anomaly score for a node exceeds a threshold, it is flagged as suspicious. ORTHRUS set the threshold to the value of the highest observed anomaly score during testing on the validation set. Then, to better differentiate between benign anomalies and malicious nodes, ORTHRUS utilizes automatic outlier clustering using a two cluster K-means clustering algorithm [4] on all suspicious nodes flagged in the thresholding step. Clusters with the highest anomaly scores are then later on prioritized by ORTHRUS, while clusters with lower scores are being treated as benign. This leads to significantly fewer false positives and thus alarms generated by ORTHRUS.

Additionally ORTHRUS allows for the reconstruction of attacks associated with malicious nodes and the generation of an attack summary graph. It achieves this by applying dependency analysis for each flagged node from the anomaly detection step. When an anomaly is detected ORTHRUS

extracts the subgraph around the anomaly within its time window. Since the dependency analysis occurs after the anomaly detection, the time window can be adjusted, and with that the information extracted, and multiple different subgraphs with varying degree of information can be extracted. ORTHRUS applies a default time window of 15 minutes. After the time window subgraph is extracted, ORTHRUS transforms it into a directed acyclic graph (DAG) and tries to identify potential attack entry and exit nodes, marking the root and the end of a path in the DAG respectively. Exit and entry nodes are then identified by applying backward and forward tracing for each DAG of a malicious node.

CAPTAIN: The alarm generation rules determine whether an alarm should be triggered.

The amount of alarms could be controlled by adjusting the threshold. The alert threshold can be fine-tuned with the help of the adaptive parameter T. While a high threshold encourages the system to emit the alarms, a low threshold stops the system from doing so. This T enables CAPTAIN to modify the edge-level alarm-triggering rules.

T can be refined through the training process. As aforementioned in [Section 3.4.2](#), $\text{thr}_i = 0.5$ is set for every edge e_i so that all alarms will not be encouraged or suppressed.

4 Analysis

IDS	Data Source	Redundancy Reduction	Edge Featurization	Graph Learning	Notable Features
PROGRAPHER	Files, processes, network sockets, etc. for nodes, and log events for edges	✓	Graph2Vec	Encoder-only	Log-focused modeling
FLASH	System logs, which consist of host event records like process executions, file operations, network connections for nodes, and system calls for edges	✓	Modified Word2Vec	GNN	Lightweight classifier, selective traversal
KAIROS	processes, files, sockets for nodes, and system events for edges	✓	Hierarchical feature hashing	Encoder-Decoder architecture	Temporal abstraction
MAGIC	Processes, files, and sockets for nodes, and system interactions for edges	✓	Masked graph representation learning	GAT	Attention mechanism
ProcSAGE	processes and threads in host audit logs	✗	✗	GraphSAGE	Focus on processes and threads
ORTHRUS	Processes, files, netflows for nodes, and system events for edges	✓	Word2Vec	Encoder-Decoder architecture, encoder also uses GNN architecture	Message passing
R-CAID	File, process, or network socket	✗	Doc2Vec + One-hot	GNN	Custom node types
CAPTAIN	audit logs (file path, process name, command line, IP address, port, etc. for nodes; and interactions for edges)	✗	Designed data/code tags, adaptive parameters	Differentiable tag propagation framework	Trust-aware inference, rule tuning

Table 1: IDS overview.

Overall While most IDS in our scope employ graph learning and embedding-based edge features, they differ significantly in redundancy reduction and system priorities. CAPTAIN, as a lightweight rule-based system, contrasts with approaches such as KAIROS, ORTHRUS, and FLASH by trading advanced multi-stage or adversarial robustness for interpretability and efficiency. Furthermore, by the choice of data source, we can define the scope of attacks an IDS can detect and also determine its detection granularity. For instance, ProcSAGE relies solely on process or thread data and can only flag suspicious processes or threads, which makes suspicious processes the primary detection result, while those that also model file and network activities can reconstruct fine-grained attack chains across processes, files, and sockets. Thus, broader data sources enable richer and more precise intrusion attribution.

4.1 Advantages

PROGRAPHER was one of the first systems to utilize the combination of Graph2Vec graph embeddings and the TextRCNN embeddings for sequence learning and indicator extraction at the graph level. This led to a high precision and accuracy for detecting APT attacks at the time. Additionally, extracting temporal-ordered snapshots from logs rather than the full provenance graph, **PROGRAPHER** reduces computational and memory costs associated with analyzing the entire provenance graph, allowing for faster detection and indicator generation making it capable of handling large-scale graphs and complex environments efficiently.

FLASH can develop a strong understanding of baseline benign behavior, by using a GNN along with temporally sensitive node embeddings, which enables **FLASH** to effectively flag even the most stealthy attack graphs. Furthermore, **FLASH** stores embeddings combined with Word2Vec vectors during the training time, which can be used as feature vectors for the XGBoost model, allowing **FLASH** to carry out anomaly detection. Moreover, by using logs from multiple hosts at once, the inference overhead per host decreases, which enables **FLASH** to save hours of processing time in large enterprise environments, mitigating log congestion and ensuring the delivery of timely alerts.

KAIROS can differentiate between benign and malicious activity based on both structural and temporal interactions between system entities. So to attack on such IDS, the adversary needs to have a great knowledge of the target's benign behavior, or even understanding of how the detection model itself operates. **KAIROS** is able to precisely reconstruct attack traces, thereby minimizing the number of graph edges that system administrators need to examine to analyze an incident. **MAGIC** uses masked graph representation learning with a self-supervised approach. It does not need labeled attack data for training, which makes it highly effective against zero-day attacks and concept drift. The system can perform detection at both batch log and node level and it adapts over time through a feedback mechanism, helping to reduce false positives.

ProcSAGE reduces graph complexity by keeping the focus on processes and threads as central nodes, while still considering other related entities like files and sockets. This keeps the graph manageable even in very large environments, while preserving key context.

R-CAID Current technique using pretrained graph embedding techniques are unable to accurately attribute attacks to their root cause. Many approaches use techniques like Continuous Bag-of-Words (CBOW), which aggregate local neighborhood information by summing over node contexts, causing a loss of non-local and structural graph relationships. These long-range, root-cause relationships are essential for root-cause analysis (RCA). Losing them during embedding therefore seriously undermines detection and enables mimicry-based evasion. This results in unutilized usage of the RCA

capabilities of provenance graphs and also enables mimicry and evasion attacks, due to the loss of long-range, root-cause relationships and global graph context that link suspicious behaviors back to immutable entry points, allowing attackers to blend into local neighborhoods and hide true lineage. Instead **R-CAID** precomputes the root cause for each node during the graph construction, avoiding having to map each node to its entire provenance history and thus costly computations, allowing for effective and efficient Root cause analysis with improved detection and resilience to evasion attacks. **R-CAID** achieves good evasion attack resistance by not relying on a loss function for the identification of anomalies combined with the power of Root cause embedding. Instead **R-CAID** detects anomalies by clustering node embeddings generated by the GNN. This way the adversary can not manipulate the loss function and avoid detection.

The previous approaches can achieve good attack detection, but they tend to overestimate the nodes involved in an attack, leading to overwhelming false positive rates and alarm fatigue. They often struggle to differentiate between malicious and benign nodes, resulting in coarse flagging and failing to attribute malicious behavior to the nodes actually responsible for the attack. For example **FLASH** and **MAGIC** often flag large regions of the graph around anomalies, producing excessive alarm data and false positives. While **KAIROS** flags all nodes occurring concurrently with an attack. This leads to an overestimation of malicious nodes, since events occurring simultaneously with an attack are treated as malicious. While **R-CAID** excels in root-cause attribution, this approach can lead to an over-estimation of the attack's scope, since every node that is a descendant of the attack's source node is incorrectly labelled as malicious.

ORTHRUS tries to mitigate alarm fatigue by providing granular, node-level detection of malicious activity across the entire provenance graph. To achieve this, **ORTHRUS** employs a two-step anomaly detection method. First, it applies an automatic thresholding technique to detect abnormal nodes. Then, to better differentiate between benign anomalies and malicious nodes, **ORTHRUS** utilizes automatic outlier clustering. This approach mitigates broad neighbourhood labelling, avoiding large number of false positives and thus resulting in increased precision. Additionally, it removes the need for manual threshold setting, improving adaptability to new environments and reducing the need for updating the thresholding criteria. Another benefit of outlier clustering is that **ORTHRUS** effectively manages class imbalances. By scoring malicious nodes as distinct outliers, **ORTHRUS** can apply tighter and more stable thresholds without compromising true positive (TP) recall. This capability is particularly crucial in imbalanced scenarios, where using loose thresholds can lead to a high number of false positives (FP).

CAPTAIN is designed as a lightweight, rule-based IDS that focuses on reducing false positives. To achieve this, it combines the classic rule-based approach with an adaptive ele-

ment that penalizes false alarms. The main advantage of this approach is its efficiency. Unlike embedding-based IDS, it requires no buffer time and only minimal pre-processing time, as it does not need to prepare the data for machine learning. The same applies to resource consumption, which is also significantly lower than with most embedding-based IDS.

4.2 Limitations

PROGRAPHER Compared to other IDS systems like FLASH, ORTHRUS or R-CAID, PROGRAPHER lacks semantic encoding and exact root cause attribution. Instead it only ranks nodes based on their anomaly score, without attributing the exact cause, which necessitates analysis for manual labour intensive analysis. Additionally PROGRAPHER is vulnerable to mimicry attacks and not immune to training contamination risks, i.e., if the training set contains undetected malicious activity, the model may learn it as normal, reducing detection effectiveness in production.. Furthermore compared PROGRAPHER is prone to false positives due to changes in normal system patterns, requiring for periodic retraining to combat concept drift.

FLASH has three main problems: False positives in the presence of unobserved benign activities, depending on enriched system logs, misclassifications in FLASH can lead to misleading AEGs, and concept drift.

KAIROS: KAIROS’ temporal encoder-decoder introduces high computational cost. Moreover in highly dynamic environments where benign behavior frequently changes, it can also trigger more false positives.

R-CAID To combat excessive graph growth, R-CAID uses a root pruning mechanism. This technique can be manipulated by an attacker, by simply attaching a malicious node to a large number of benign nodes, causing the malicious node to be pruned. Additionally, as mentioned earlier, struggles with the over-estimation of an attack’s scope.

ORTHRUS still produces false positives on certain datasets, usually within the one-neighborhood of a malicious node due to the nature of GNNs. GNN message passing can propagate malicious signals to neighboring nodes, producing false positives in the attack vicinity. The performance of ORTHRUS, like many provenance-based intrusion detection systems, is significantly impacted by the underlying capture mechanisms used to record system events. Some systems omit critical details, such as CADETS frequently missing file and process paths, while CLEARSCOPE lacks comprehensive information like process paths and remote netflow addresses, resulting in degraded performance. Furthermore ORTHRUS needs periodic retraining as software and benign behavior change, changes of the underlying patterns and relationships in the system. Resulting in initially trained models becoming less effective at distinguishing between benign and malicious activities and thus a increased FP rate. Nonetheless, computational efficiency of ORTHRUS allows

for frequency retraining, even if not ideal.

CAPTAIN has due to its rule-based approach problems to handle unknown behavior. Furthermore it is less scalable than embedding systems like MAGIC, which perform redundancy reduction.

4.3 Datasets

To train and evaluate the IDS, the paper use different dataset, which provide large-scale provenance data of host systems under benign and malicious workload. Each Dataset consists of detailed system-level audit logs, including processes, files and network activities. Most common are the E3 and E5 of DARPA’s Transparent computing program. The E3 is smaller and less complex, while E5 contains more advanced attack data. In the following table, we will see the evaluation datasets used by provenance-based IDS systems.

System	CADETS	THEIA	TRACE
PROGRAPHER	✓(E3)	✓(E3)	✗
FLASH	✓(E3)	✗	✓(E3)
KAIROS	✓(E3)	✓(E3)	✓(E3)
MAGIC	✓(E3)	✓(E3)	✓(E3)
ProcSAGE	✗	✗	✗
R-CAID	✗	✓(E3)	✓(E3)
ORTHRUS	✓(E3, E5)	✓(E3, E5)	✗
CAPTAIN	✓(E3)	✓(E3)	✓(E3)

Table 2: Evaluation datasets

4.4 Attack resistance

We want to compare, which IDS are optimized for which attack detection categories. Therefore we selected three common threat types.

System	Detect zero-Day Exploits	vulnerable to Mimicry attacks	APT detection
PROGRAPHER	✓	✓	✓
FLASH	✓	✗	✓
KAIROS	✓	~	✓
MAGIC	✓	~	✓
ProcSAGE	~	✗	✓
R-CAID	✓	✗	✓
ORTHRUS	✓	✗	✗
CAPTAIN	✓	✗	✓

Table 3: Attack resistance of the IDS

Zero-Day Exploits are attacks that take advantage of software vulnerabilities before they are publicly known and patched. For many IDS this is a problem, because they are

trained on preexisting attack data or are rule based. But most of IDS in our scope can detect this kind of attack, since they learn only benign behavior without any predefined attacks, or malicious knowledge.

Mimicry/Evasion are designed to hide malicious behavior by imitating the patterns of normal system activity. Evasion describes attacks in which the attacker uses his knowledge of an IDS to introduce noise or mimic benign system behavior to mislead the detection System. R-CAID, PROGRAPHER and FLASH are optimized, because they include more context and relationship into their model, which is hard to imitate.

APT detection are attacks that occur over a longer period of time and in several phases. Most of the systems are good in recognizing these attacks. Therefore the systems, which capture temporal development and chains of actions, are particularly suitable.

Conclusion: Most of the aforementioned IDS are well-designed for detecting these three increasingly common attacks, but they still have some limitations, which are our future works to improve them. For example, some IDS have limitations against zero-day exploits and mimicry/evasion attacks since they lack explicit mechanisms for tracking APT progression across time, or key mechanisms. Some of them can not against some advanced mimicry/evasion attacks such as adversarially crafted attacks because they do not introduce perturbed malicious samples during training.

4.5 Use cases

The systems differ also in their optimal use cases. Depending on how much a system's utilization fluctuates, it is referred to as either **static** or **dynamic**. In static environments processes and workloads change only little. This is beneficial for IDS, as malicious behavior deviates from the pattern and is therefore easy to detect. In dynamic environments, processes and workloads are constantly changing, which means that IDS must be adaptive. In some cases benign behavior changes over time, which means IDS must be adaptive in their anomaly detection.

System	Static/Dynamic	Small/Large	Evolving
PROGRAPHER	Dynamic	Large	✓
FLASH	Both	Both	✗
KAIROS	Both	Both	✗
MAGIC	Dynamic	Large	✓
ProcSAGE	Static	esp. Large	✗
R-CAID	Dynamic	Small	✓
ORTHRUS	Dynamic	Large	✓
CAPTAIN	Static	Both	✓

Table 4: Use cases of the IDS

Most of the systems are well suited for large dynamic systems. Thanks to their architecture, Kairos and ProcSAGE are particularly scalable. This makes them perfect for very large networks.

PROGRAPHER uses a snapshot-based model, which makes it ideal for evolving environments. Captain also has adaptive parts and MAGIC uses the Masked Graph Learning, which increases their performance in such cases.

References

- [1] Lirim Ashiku and Cihan Dagli. Network Intrusion Detection System using Deep Learning. *Procedia Computer Science*, 185:239–247, 2021. Publisher: Elsevier BV.
- [2] Zijun Cheng, Qiujuan Lv, Jinyuan Liang, Yan Wang, Degang Sun, Thomas Pasquier, and Xueyuan Han. Kairos: Practical Intrusion Detection and Investigation using Whole-system Provenance. In *2024 IEEE Symposium on Security and Privacy (SP)*, pages 3533–3551, May 2024. ISSN: 2375-1207.
- [3] Akul Goyal, Gang Wang, and Adam Bates. R-CAID: Embedding Root Cause Analysis within Provenance-based Intrusion Detection. In *2024 IEEE Symposium on Security and Privacy (SP)*, pages 3515–3532, May 2024. ISSN: 2375-1207.
- [4] J. A. Hartigan and M. A. Wong. Algorithm AS 136: A K-Means Clustering Algorithm. *Journal of the Royal Statistical Society. Series C (Applied Statistics)*, 28(1):100–108, 1979. Publisher: [Royal Statistical Society, Oxford University Press].
- [5] Stamus Networks Inc. IDS Detection Types.
- [6] Zian Jia and Yun Xiong. MAGIC: Detecting Advanced Persistent Threats via Masked Graph Representation Learning.
- [7] Baoxiang Jiang, Tristan Bilot, Nour El Madhoun, Khalidoun Al Agha, Anis Zouaoui, Shahrear Iqbal, Xueyuan Han, and Thomas Pasquier. ORTHRUS: Achieving High Quality of Attribution in Provenance-based Intrusion Detection Systems.
- [8] Merve Ozkan-Okay, Refik Samet, Ömer Aslan, and Deepti Gupta. A Comprehensive Systematic Literature Review on Intrusion Detection Systems. *IEEE Access*, 9:157727–157760, 2021.
- [9] Karen Scarfone and Peter Mell. Guide to Intrusion Detection and Prevention Systems (IDPS). Technical Report NIST Special Publication (SP) 800-94, National Institute of Standards and Technology, February 2007.
- [10] Mati Ur Rehman, Hadi Ahmadi, and Wajih Ul Hassan. Flash: A Comprehensive Approach to Intrusion Detection via Provenance Graph Representation Learning. In *2024 IEEE Symposium on Security and Privacy (SP)*, pages 3552–3570, May 2024. ISSN: 2375-1207.
- [11] Lingzhi Wang, Xiangmin Shen, Weijian Li, Zhenyuan Li, R. Sekar, Han Liu, and Yan Chen. Incorporating Gradients to Rules: Towards Lightweight, Adaptive Provenance-based Intrusion Detection. In *Proceedings 2025 Network and Distributed System Security Symposium*, San Diego, CA, USA, 2025. Internet Society.
- [12] Boyuan Xu, Yiru Gong, Xiaoyu Geng, Yun Li, Cong Dong, Song Liu, Yuling Liu, Bo Jiang, and Zhigang Lu. ProcSAGE: an efficient host threat detection method based on graph representation learning. *Cybersecurity*, 7(1):51, August 2024.
- [13] Fan Yang, Jiachen Xu, Chunlin Xiong, Zhou Li, and Kehuan Zhang. {PROGRAPHER}: An Anomaly Detection System based on Provenance Graph Embedding. pages 4355–4372, 2023.
- [14] Michael Zipperle, Florian Gottwalt, Elizabeth Chang, and Tharam Dillon. Provenance-based Intrusion Detection Systems: A Survey. *ACM Comput. Surv.*, 55(7):135:1–135:36, December 2022.